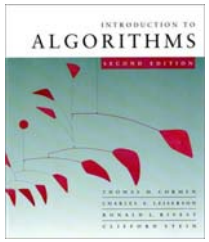
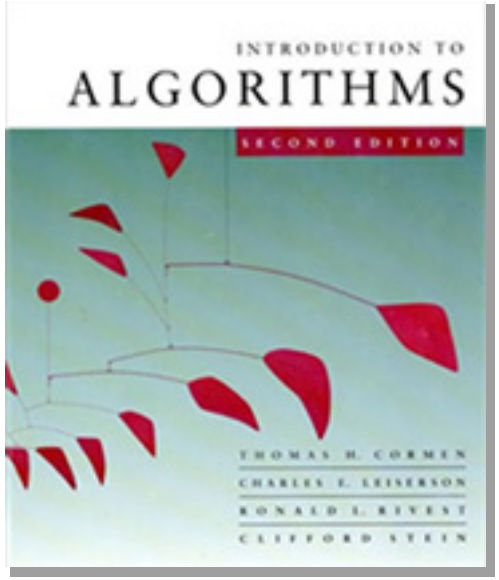


Introduction to Algorithms

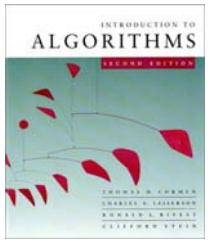


LECTURE 15

Shortest Paths I

- Properties of shortest paths
- Dijkstra's algorithm
- Correctness
- Analysis
- Breadth-first search

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



Paths in graphs

Consider a digraph $G = (V, E)$ with edge-weight function $w : E \to \mathbb{R}$. The ***weight*** of path $p = v_1 \to v_2 \to \cdots \to v_k$ is defined to be

$$w(p) = \sum_{i=1}^{k-1} w(v_i, v_{i+1}).$$

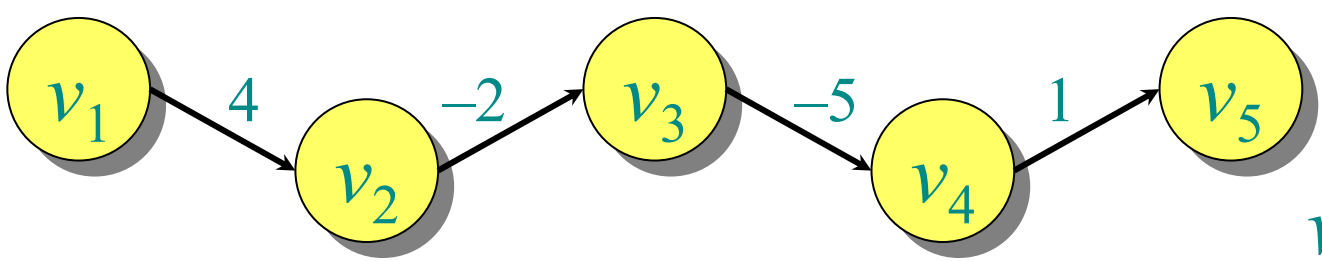
 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

Paths in graphs

Consider a digraph $G = (V, E)$ with edge-weight function $w : E \to \mathbb{R}$. The ***weight*** of path $p = v_1 \to v_2 \to \cdots \to v_k$ is defined to be

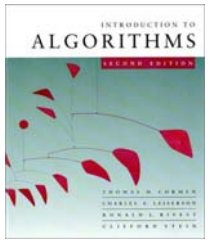
$$w(p) = \sum_{i=1}^{k-1} w(v_i, v_{i+1}).$$

Example:



$w(p) = -2$

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



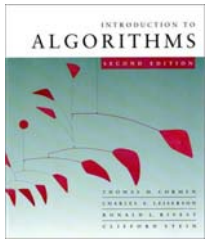
Shortest paths

A ***shortest path*** from u to v is a path of minimum weight from u to v. The ***shortest-path weight*** from u to v is defined as

$$\delta(u, v) = \min\{w(p) : p \text{ is a path from } u \text{ to } v\}.$$

Note: $\delta(u, v) = \infty$ if no path from u to v exists.

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



Optimal substructure

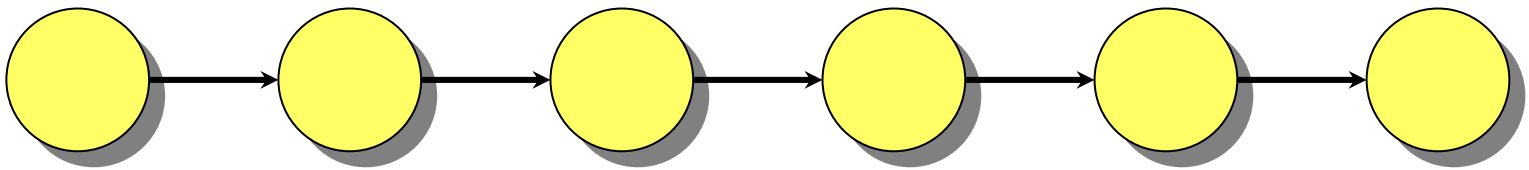
Theorem. A subpath of a shortest path is a shortest path.

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

Optimal substructure

Theorem. A subpath of a shortest path is a shortest path.

Proof. Cut and paste:

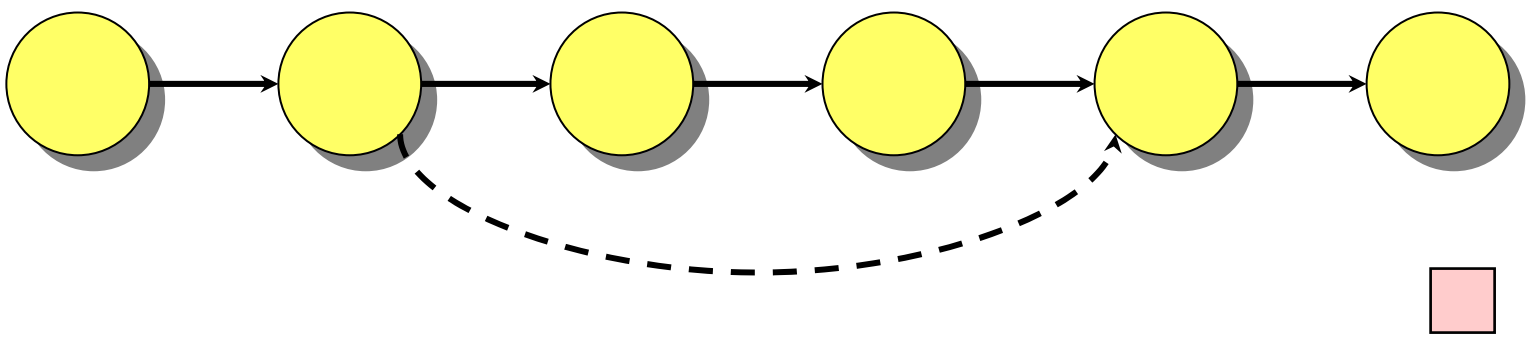


 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

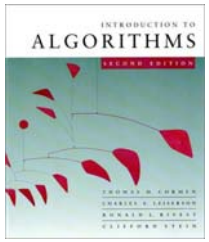
Optimal substructure

Theorem. A subpath of a shortest path is a shortest path.

Proof. Cut and paste:



 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



Triangle inequality

Theorem. For all $u, v, x \in V$, we have

$$\delta(u, v) \le \delta(u, x) + \delta(x, v).$$

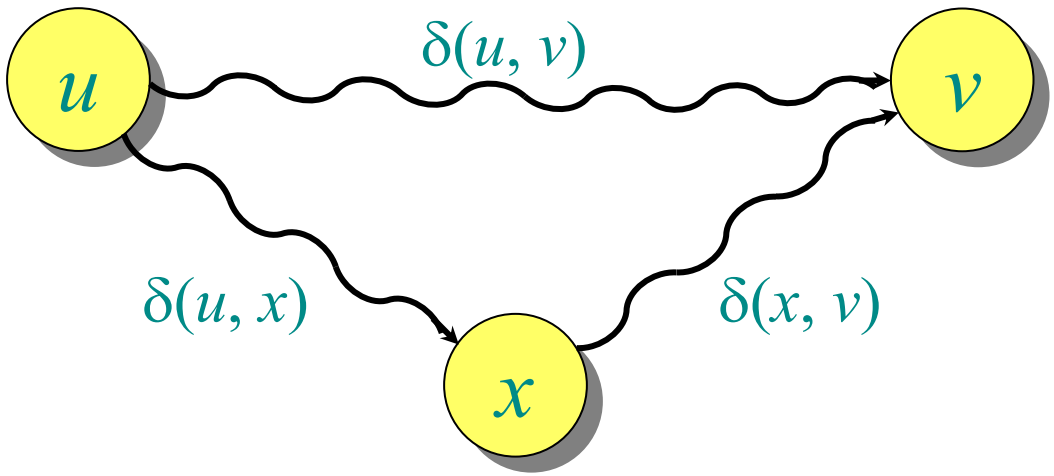
 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

Triangle inequality

Theorem. For all $u, v, x \in V$, we have

$$\delta(u, v) \le \delta(u, x) + \delta(x, v).$$

Proof.



 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

Well-definedness of shortest paths

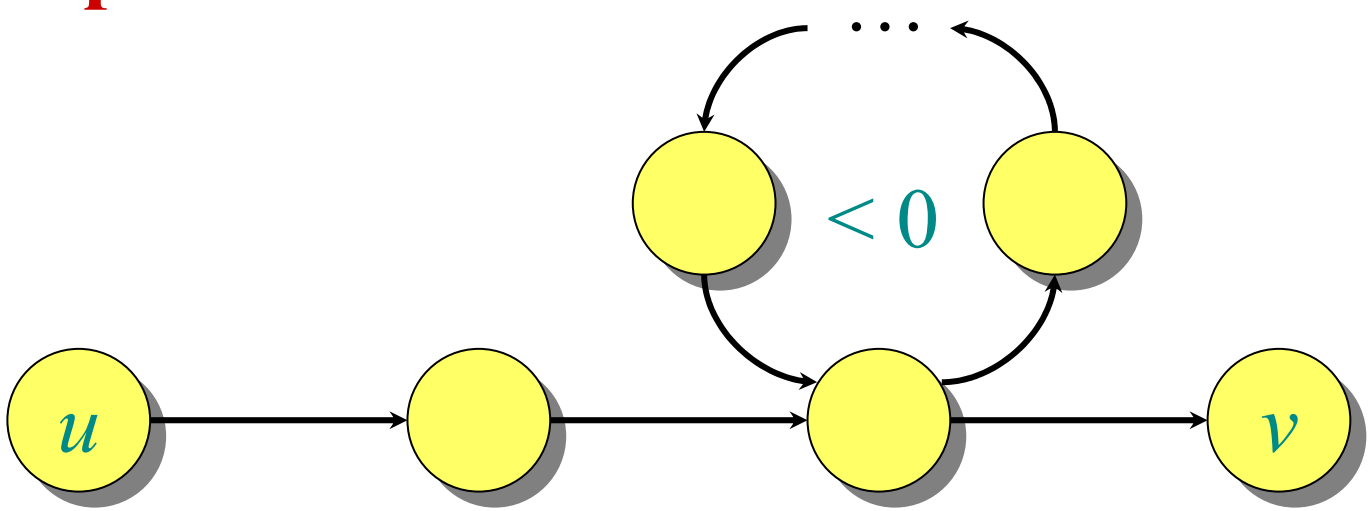
If a graph G contains a negative-weight cycle, then some shortest paths may not exist.

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

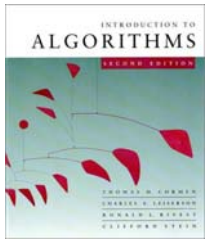
Well-definedness of shortest paths

If a graph G contains a negative-weight cycle, then some shortest paths may not exist.

Example:



 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



Single-source shortest paths

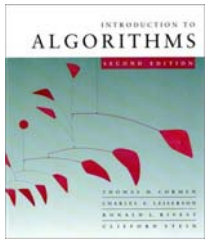
Problem. From a given source vertex $s \in V$, find the shortest-path weights $\delta(s, v)$ for all $v \in V$.

If all edge weights $w(u, v)$ are *nonnegative*, all shortest-path weights must exist.

IDEA: Greedy.

1. Maintain a set S of vertices whose shortest-path distances from s are known.
2. At each step add to S the vertex $v \in V - S$ whose distance estimate from s is minimal.
3. Update the distance estimates of vertices adjacent to v.

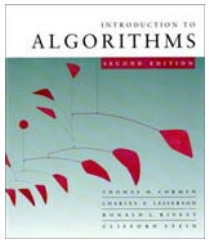
 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



Dijkstra's algorithm

$d[s] \leftarrow 0$
for each $v \in V - \{s\}$
    do $d[v] \leftarrow \infty$
$S \leftarrow \varnothing$
$Q \leftarrow V$ ▷ Q is a priority queue maintaining $V - S$

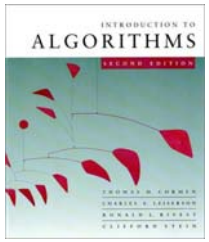
 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



Dijkstra's algorithm

$d[s] \leftarrow 0$
for each $v \in V - \{s\}$
    do $d[v] \leftarrow \infty$
$S \leftarrow \varnothing$
$Q \leftarrow V$ ▷ Q is a priority queue maintaining $V - S$
while $Q \neq \varnothing$
    do $u \leftarrow$ EXTRACT-MIN(Q)
        $S \leftarrow S \cup \{u\}$
        for each $v \in Adj[u]$
            do if $d[v] > d[u] + w(u, v)$
                then $d[v] \leftarrow d[u] + w(u, v)$

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



Dijkstra's algorithm

$d[s] \leftarrow 0$
for each $v \in V - \{s\}$
    do $d[v] \leftarrow \infty$
$S \leftarrow \varnothing$
$Q \leftarrow V$ ▷ Q is a priority queue maintaining $V - S$
while $Q \neq \varnothing$
    do $u \leftarrow$ EXTRACT-MIN(Q)
        $S \leftarrow S \cup \{u\}$
        for each $v \in Adj[u]$
            do **if** $d[v] > d[u] + w(u, v)$
                then $d[v] \leftarrow d[u] + w(u, v)$

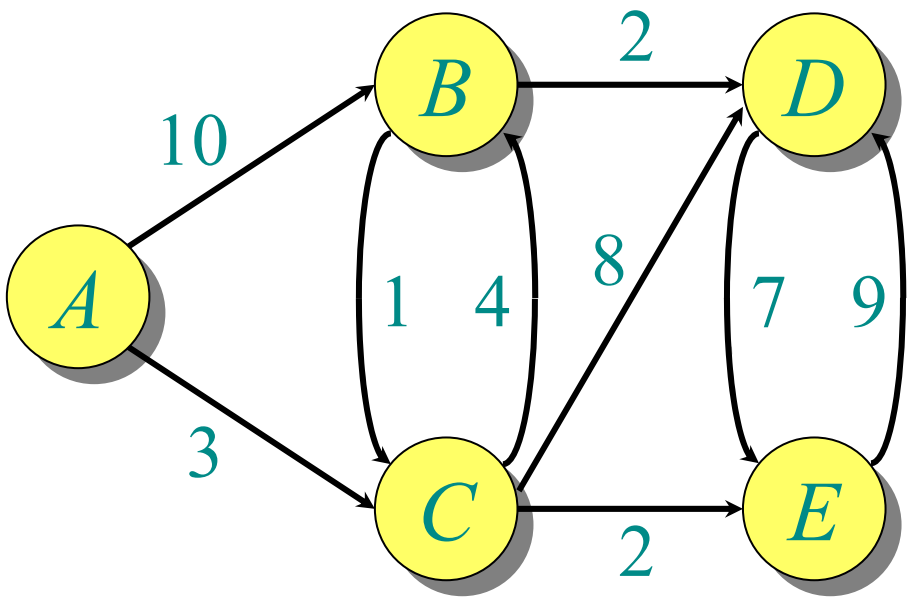
relaxation step

Implicit DECREASE-KEY

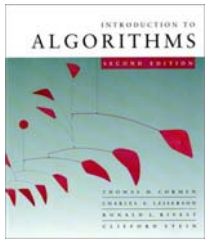
 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

Example of Dijkstra's algorithm

Graph with nonnegative edge weights:

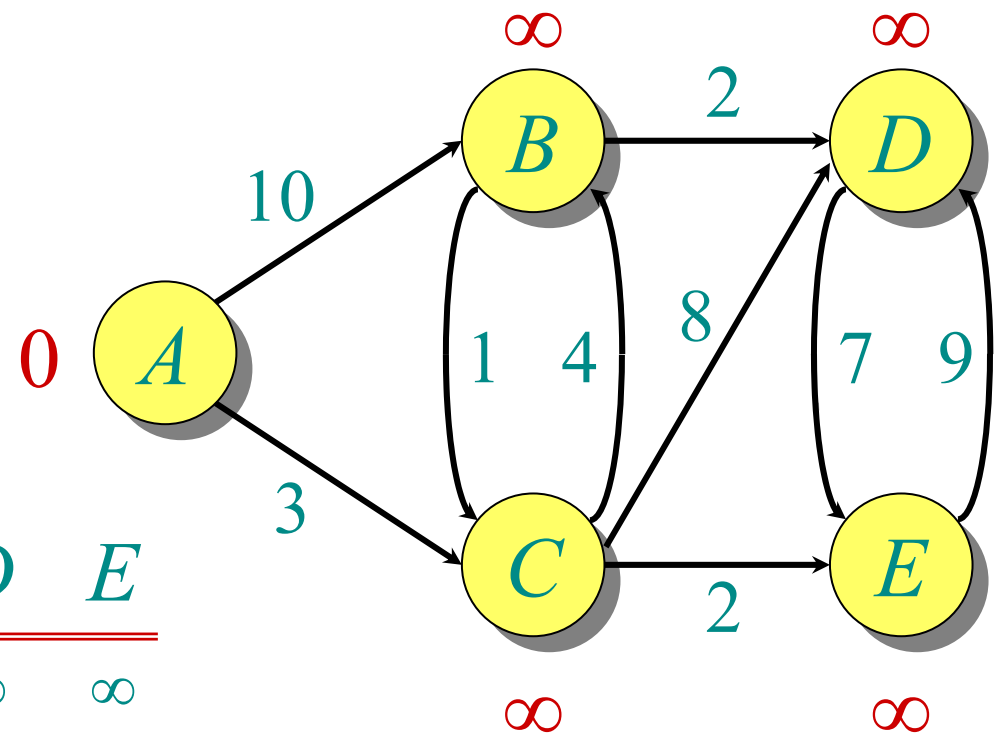


 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



Example of Dijkstra's algorithm

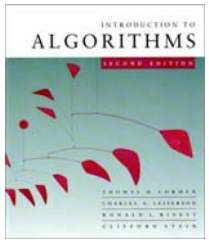
Initialize:



Q:	A	B	C	D	E
	0	∞	∞	∞	∞

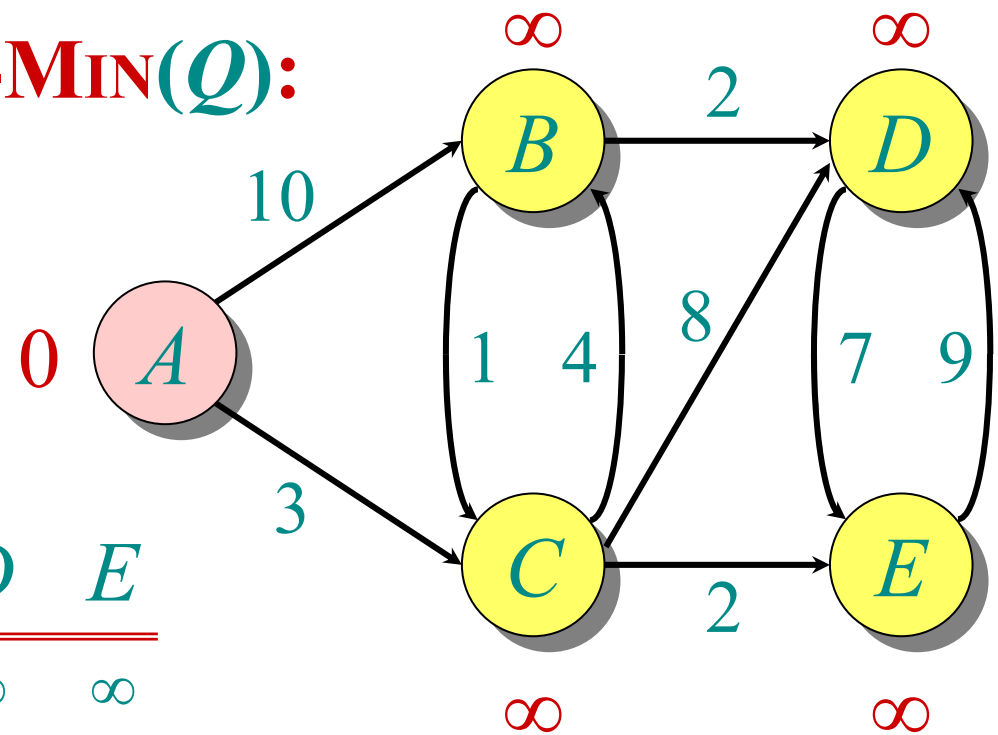
S: {}

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



Example of Dijkstra's algorithm

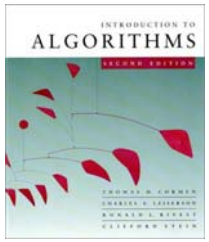
"A" ← EXTRACT-MIN(Q):



Q:	A	B	C	D	E
	0	∞	∞	∞	∞

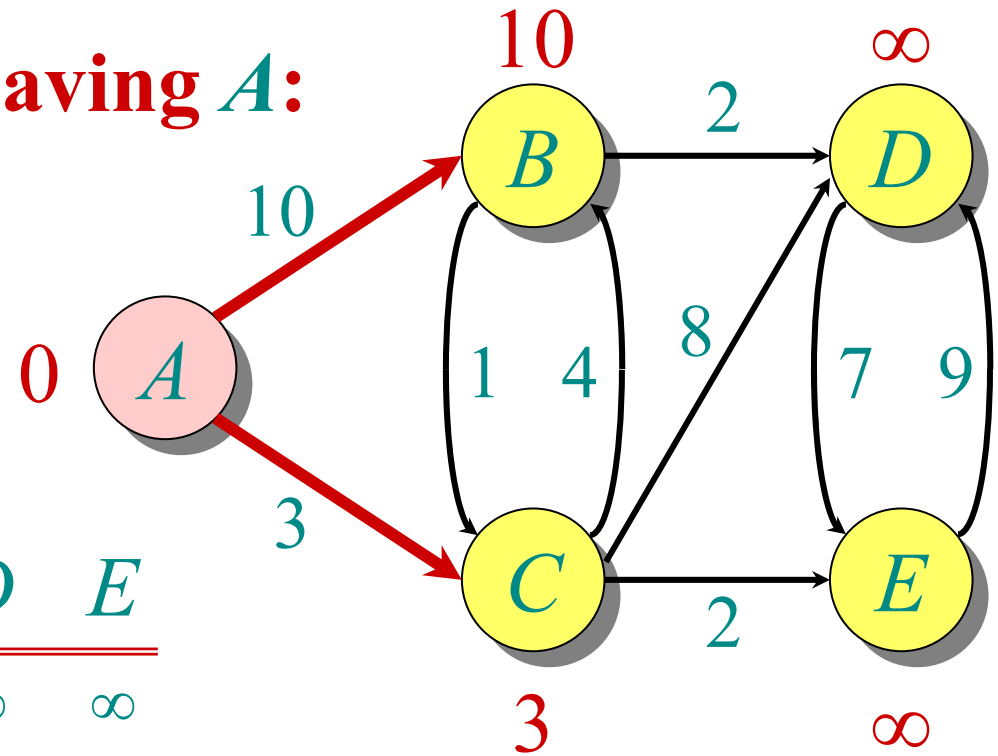
S: { A }

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



Example of Dijkstra's algorithm

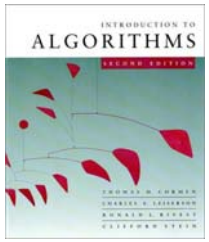
Relax all edges leaving *A*:



Q:	*A*	*B*	*C*	*D*	*E*
	0	∞	∞	∞	∞
		10	3	∞	∞

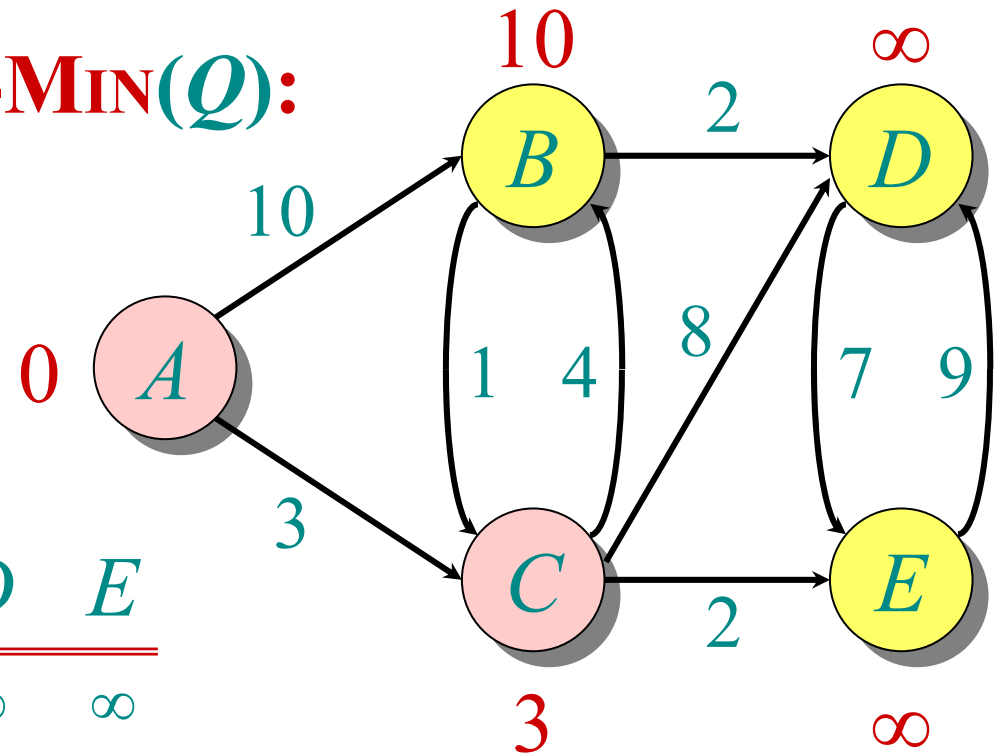
S: { *A* }

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



Example of Dijkstra's algorithm

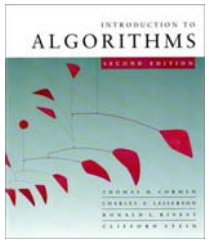
"C" ← EXTRACT-MIN(Q):



Q:	A	B	C	D	E
	0	∞	∞	∞	∞
		10	3	∞	∞

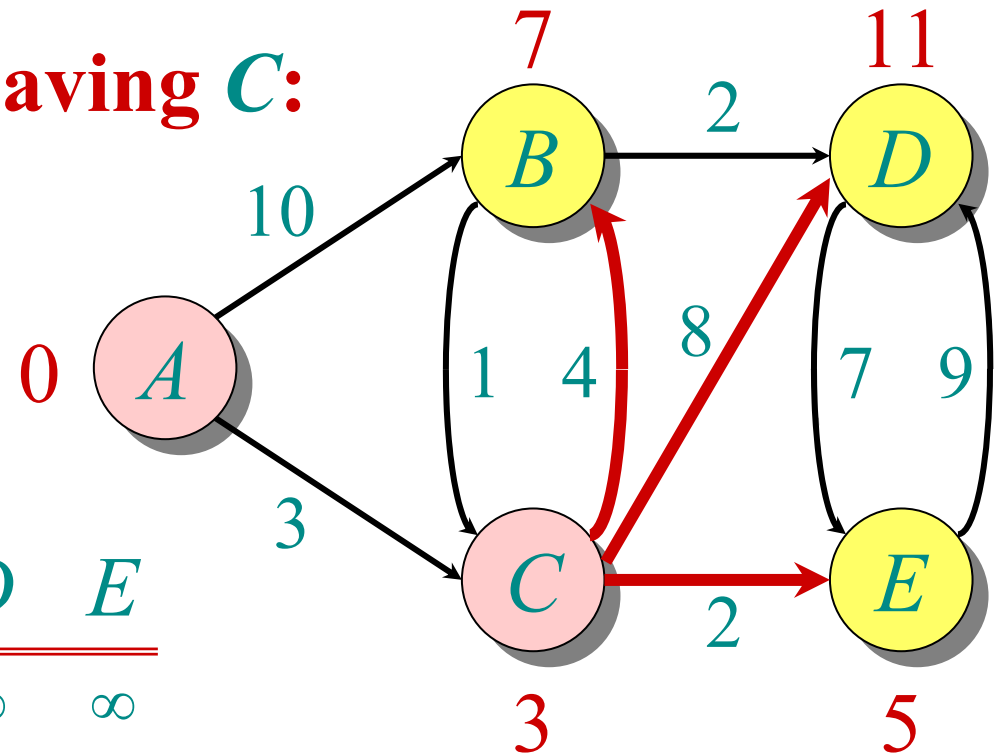
S: { A, C }

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



Example of Dijkstra's algorithm

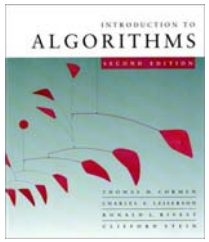
Relax all edges leaving *C*:



Q:	*A*	*B*	*C*	*D*	*E*
	0	∞	∞	∞	∞
		10	3	∞	∞
		7		11	5

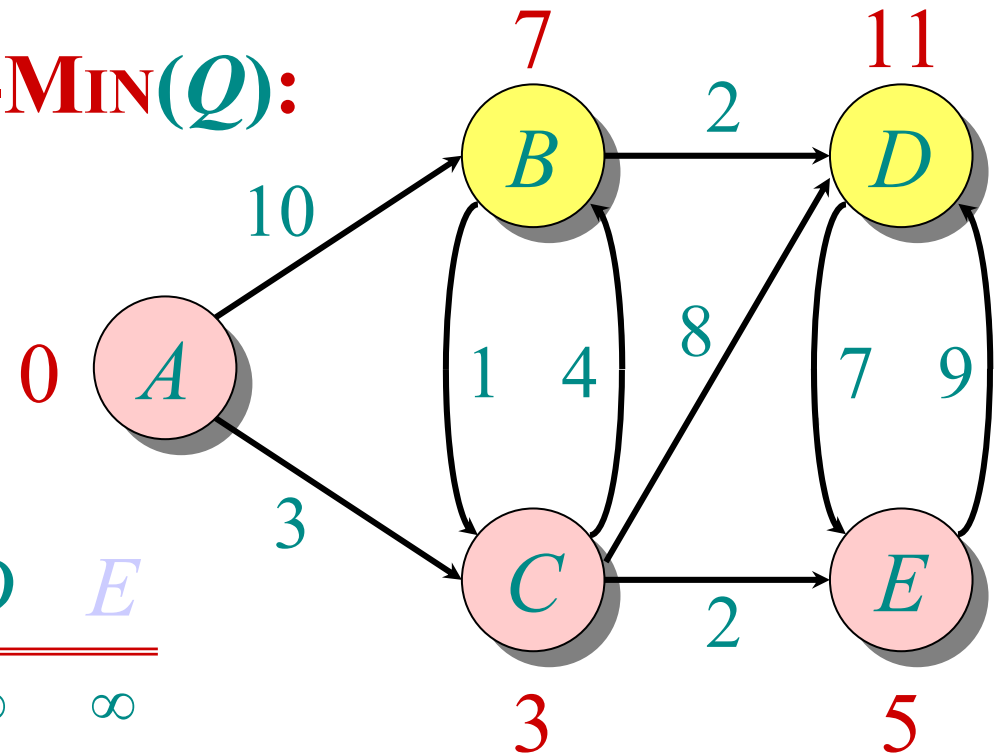
S: { *A*, *C* }

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



Example of Dijkstra's algorithm

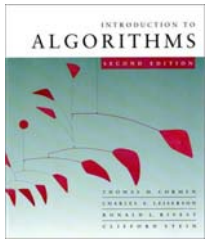
"*E*" ← EXTRACT-MIN(*Q*):



Q:	*A*	*B*	*C*	*D*	*E*
	0	∞	∞	∞	∞
		10	3	∞	∞
		7		11	5

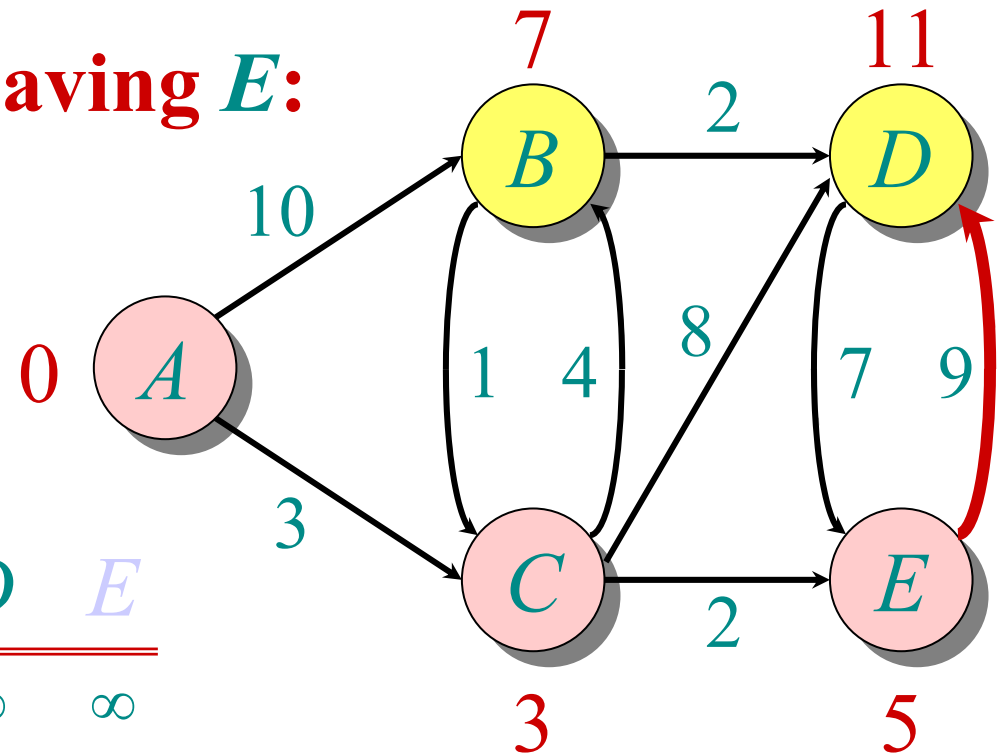
S: { *A*, *C*, *E* }

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



Example of Dijkstra's algorithm

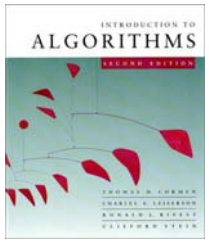
Relax all edges leaving *E*:



Q:	*A*	*B*	*C*	*D*	*E*
	0	∞	∞	∞	∞
		10	3	∞	∞
		7		11	5
		7		11	

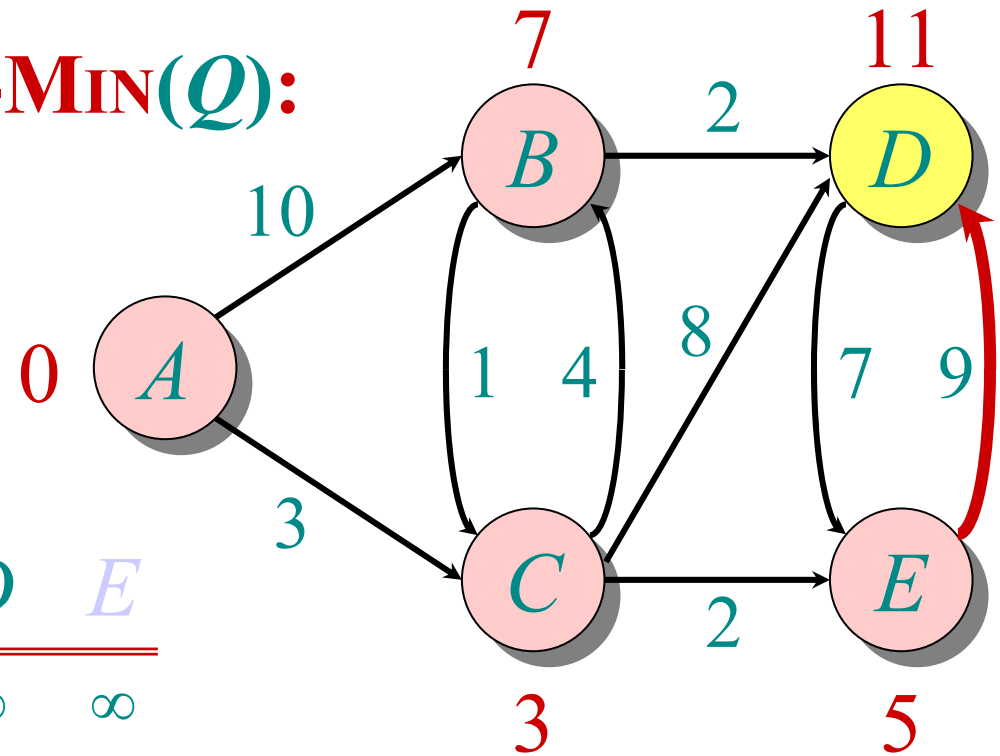
S: { *A*, *C*, *E* }

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



Example of Dijkstra's algorithm

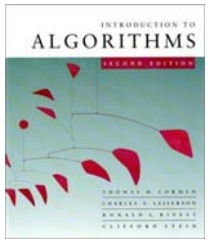
"*B*" ← EXTRACT-MIN(*Q*):



Q:	*A*	*B*	*C*	*D*	*E*
	0	∞	∞	∞	∞
		10	3	∞	∞
		7		11	5
		7		11	

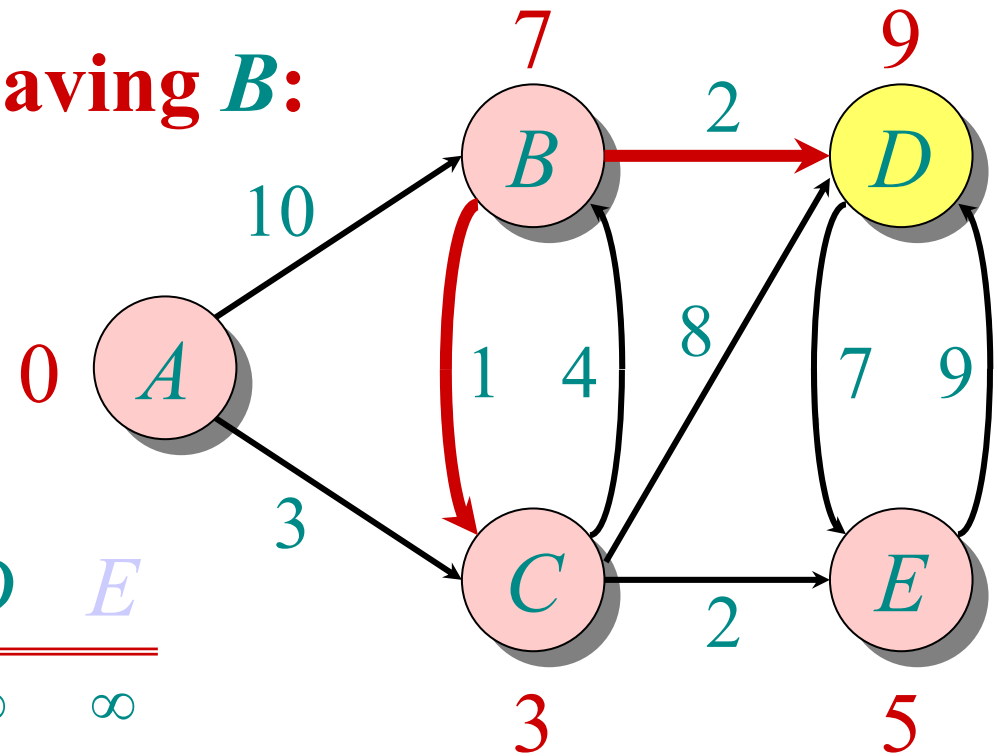
S: { *A*, *C*, *E*, *B* }

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



Example of Dijkstra's algorithm

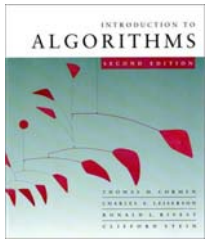
Relax all edges leaving *B*:



Q:	*A*	*B*	*C*	*D*	*E*
	0	∞	∞	∞	∞
		10	3	∞	∞
		7		11	5
		7		11	
				9	

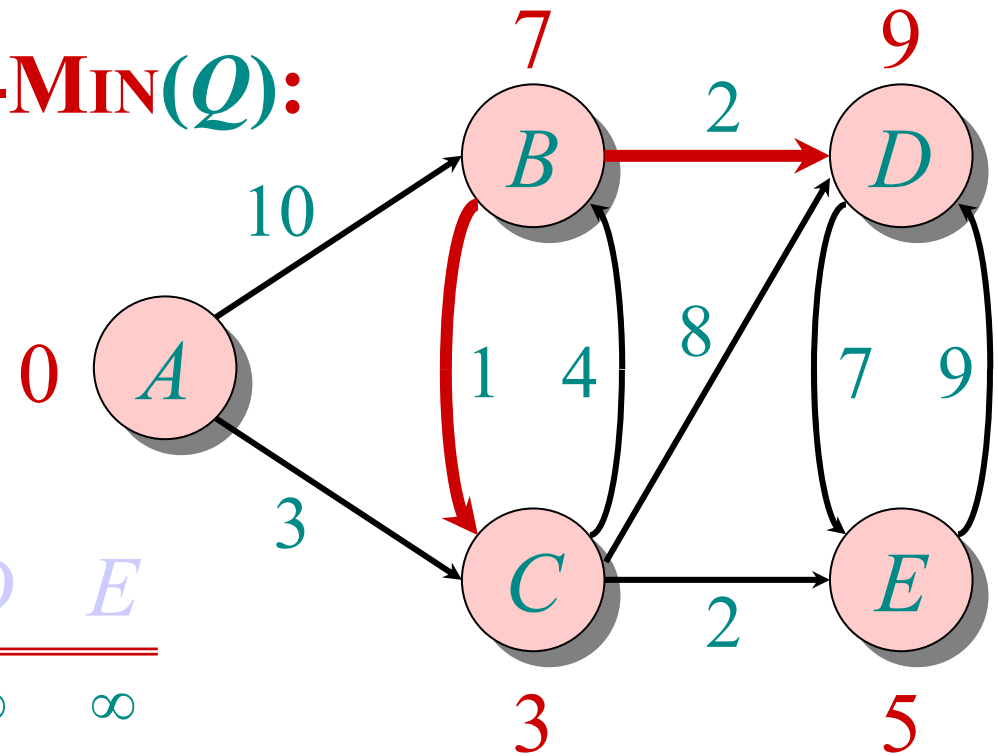
S: { *A*, *C*, *E*, *B* }

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



Example of Dijkstra's algorithm

"*D*" ← EXTRACT-MIN(*Q*):



Q:	*A*	*B*	*C*	*D*	*E*
	0	∞	∞	∞	∞
		10	3	∞	∞
		7		11	5
		7		11	
				9	

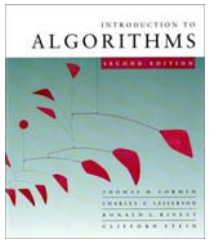
S: { *A*, *C*, *E*, *B*, *D* }

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

Correctness — Part I

Lemma. Initializing $d[s] \leftarrow 0$ and $d[v] \leftarrow \infty$ for all $v \in V - \{s\}$ establishes $d[v] \geq \delta(s, v)$ for all $v \in V$, and this invariant is maintained over any sequence of relaxation steps.

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



Correctness — Part I

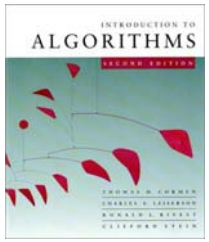
Lemma. Initializing $d[s] \leftarrow 0$ and $d[v] \leftarrow \infty$ for all $v \in V - \{s\}$ establishes $d[v] \geq \delta(s, v)$ for all $v \in V$, and this invariant is maintained over any sequence of relaxation steps.

Proof. Suppose not. Let v be the first vertex for which $d[v] < \delta(s, v)$, and let u be the vertex that caused $d[v]$ to change: $d[v] = d[u] + w(u, v)$. Then,

$d[v] < \delta(s, v)$	supposition
$\leq \delta(s, u) + \delta(u, v)$	triangle inequality
$\leq \delta(s,u) + w(u, v)$	sh. path $\leq$ specific path
$\leq d[u] + w(u, v)$	v is first violation

Contradiction. ∎

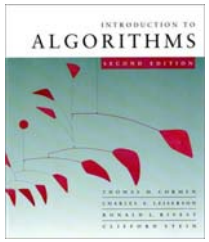
 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



Correctness — Part II

Lemma. Let u be v's predecessor on a shortest path from s to v. Then, if $d[u] = \delta(s, u)$ and edge (u, v) is relaxed, we have $d[v] = \delta(s, v)$ after the relaxation.

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

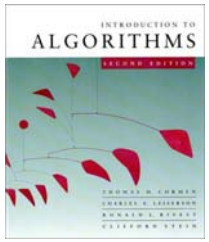


Correctness — Part II

Lemma. Let u be v's predecessor on a shortest path from s to v. Then, if $d[u] = \delta(s, u)$ and edge (u, v) is relaxed, we have $d[v] = \delta(s, v)$ which is maintained thereafter.

Proof. Observe that $\delta(s, v) = \delta(s, u) + w(u, v)$. Suppose that $d[v] > \delta(s, v)$ before the relaxation. (Otherwise, we're done.) Then, the test $d[v] > d[u] + w(u, v)$ succeeds, because $d[v] > \delta(s, v) = \delta(s, u) + w(u, v) = d[u] + w(u, v)$, and the algorithm sets $d[v] = d[u] + w(u, v) = \delta(s, v)$. ∎

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



Correctness — Part III

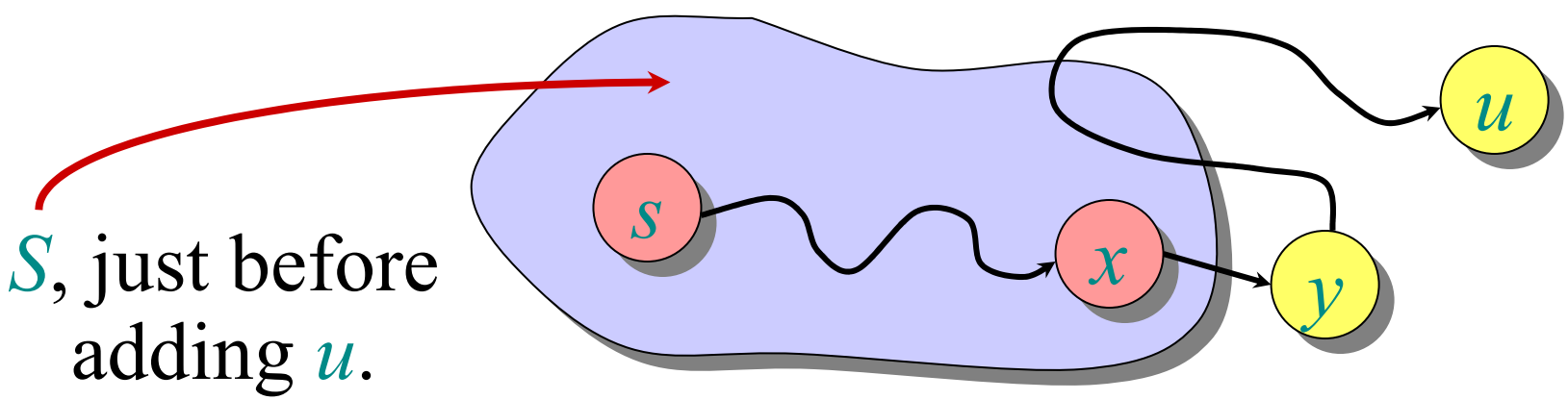
Theorem. Dijkstra's algorithm terminates with $d[v] = \delta(s, v)$ for all $v \in V$.

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

Correctness — Part III

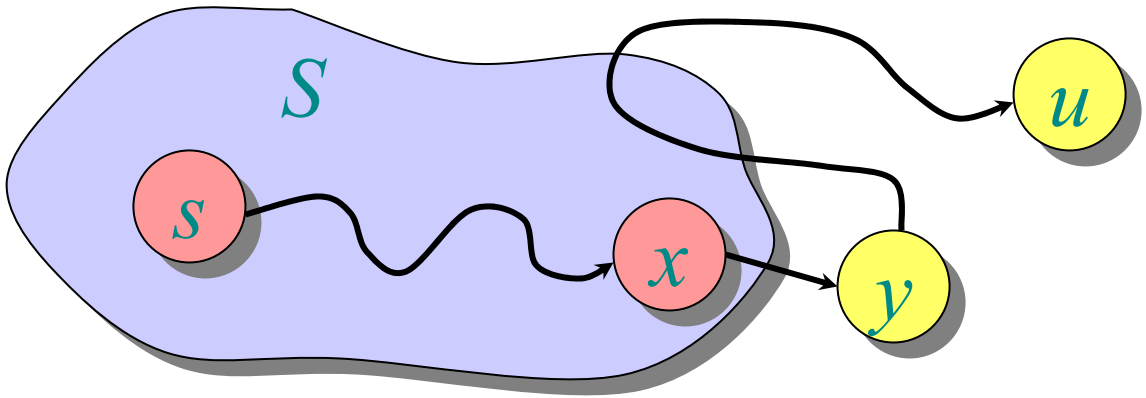
Theorem. Dijkstra's algorithm terminates with $d[v] = \delta(s, v)$ for all $v \in V$.

Proof. It suffices to show that $d[v] = \delta(s, v)$ for every $v \in V$ when v is added to S. Suppose u is the first vertex added to S for which $d[u] > \delta(s, u)$. Let y be the first vertex in $V - S$ along a shortest path from s to u, and let x be its predecessor:



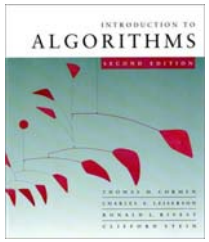
 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

Correctness — Part III (continued)



Since u is the first vertex violating the claimed invariant, we have $d[x] = \delta(s, x)$. When x was added to S, the edge (x, y) was relaxed, which implies that $d[y] = \delta(s, y) \leq \delta(s, u) < d[u]$. But, $d[u] \leq d[y]$ by our choice of u. Contradiction. ∎

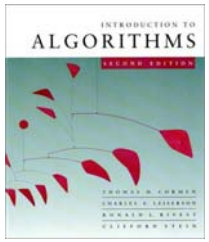
 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



Analysis of Dijkstra

while $Q \neq \varnothing$
  do $u \leftarrow$ EXTRACT-MIN(Q)
    $S \leftarrow S \cup \{u\}$
    for each $v \in Adj[u]$
      do if $d[v] > d[u] + w(u, v)$
        then $d[v] \leftarrow d[u] + w(u, v)$

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

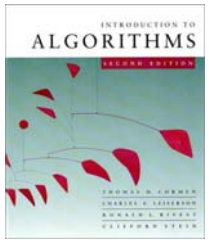


Analysis of Dijkstra

$|V|$ times

while $Q \neq \varnothing$
  do $u \leftarrow$ EXTRACT-MIN(Q)
    $S \leftarrow S \cup \{u\}$
    for each $v \in Adj[u]$
      do if $d[v] > d[u] + w(u, v)$
        then $d[v] \leftarrow d[u] + w(u, v)$

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



Analysis of Dijkstra

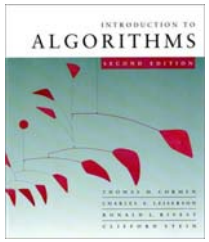
$|V|$ times

while $Q \neq \varnothing$
  do $u \leftarrow$ EXTRACT-MIN(Q)
    $S \leftarrow S \cup \{u\}$

$degree(u)$ times

    for each $v \in Adj[u]$
      do if $d[v] > d[u] + w(u, v)$
        then $d[v] \leftarrow d[u] + w(u, v)$

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



Analysis of Dijkstra

$|V|$ times

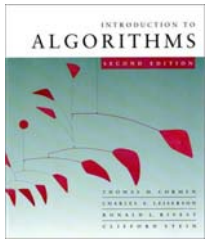
while $Q \neq \varnothing$
  do $u \leftarrow$ EXTRACT-MIN(Q)
    $S \leftarrow S \cup \{u\}$

$degree(u)$ times

    for each $v \in Adj[u]$
      do if $d[v] > d[u] + w(u, v)$
        then $d[v] \leftarrow d[u] + w(u, v)$

Handshaking Lemma $\Rightarrow$ $\Theta(E)$ implicit DECREASE-KEY's.

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



Analysis of Dijkstra

$|V|$ times:

while $Q \neq \varnothing$
  do $u \leftarrow$ EXTRACT-MIN(Q)
    $S \leftarrow S \cup \{u\}$

$degree(u)$ times:

    for each $v \in Adj[u]$
      do if $d[v] > d[u] + w(u, v)$
        then $d[v] \leftarrow d[u] + w(u, v)$

Handshaking Lemma $\Rightarrow \Theta(E)$ implicit DECREASE-KEY's.

Time $= \Theta(V \cdot T_{\text{EXTRACT-MIN}} + E \cdot T_{\text{DECREASE-KEY}})$

Note: Same formula as in the analysis of Prim's minimum spanning tree algorithm.

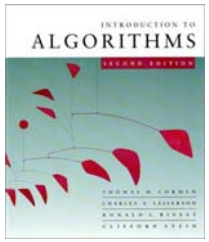
 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

Analysis of Dijkstra (continued)

$$\text{Time} = \Theta(V)\cdot T_{\text{EXTRACT-MIN}} + \Theta(E)\cdot T_{\text{DECREASE-KEY}}$$

Q	$T_{\text{EXTRACT-MIN}}$	$T_{\text{DECREASE-KEY}}$	Total

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



Analysis of Dijkstra (continued)

$$\text{Time} = \Theta(V)\cdot T_{\text{EXTRACT-MIN}} + \Theta(E)\cdot T_{\text{DECREASE-KEY}}$$

Q	$T_{\text{EXTRACT-MIN}}$	$T_{\text{DECREASE-KEY}}$	Total
array	$O(V)$	$O(1)$	$O(V^2)$

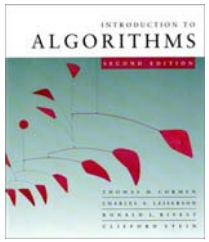
 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

Analysis of Dijkstra (continued)

$$\text{Time} = \Theta(V)\cdot T_{\text{EXTRACT-MIN}} + \Theta(E)\cdot T_{\text{DECREASE-KEY}}$$

Q	$T_{\text{EXTRACT-MIN}}$	$T_{\text{DECREASE-KEY}}$	Total
array	$O(V)$	$O(1)$	$O(V^2)$
binary heap	$O(\lg V)$	$O(\lg V)$	$O(E \lg V)$

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



Analysis of Dijkstra (continued)

$$\text{Time} = \Theta(V)\cdot T_{\text{EXTRACT-MIN}} + \Theta(E)\cdot T_{\text{DECREASE-KEY}}$$

Q	$T_{\text{EXTRACT-MIN}}$	$T_{\text{DECREASE-KEY}}$	Total
array	$O(V)$	$O(1)$	$O(V^2)$
binary heap	$O(\lg V)$	$O(\lg V)$	$O(E \lg V)$
Fibonacci heap	$O(\lg V)$ amortized	$O(1)$ amortized	$O(E + V \lg V)$ worst case

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

Unweighted graphs

Suppose that $w(u, v) = 1$ for all $(u, v) \in E$.
Can Dijkstra's algorithm be improved?

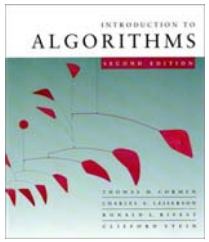
 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

Unweighted graphs

Suppose that $w(u, v) = 1$ for all $(u, v) \in E$.
Can Dijkstra's algorithm be improved?

- Use a simple FIFO queue instead of a priority queue.

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



Unweighted graphs

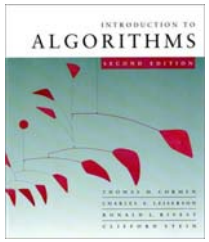
Suppose that $w(u, v) = 1$ for all $(u, v) \in E$.
Can Dijkstra's algorithm be improved?

- Use a simple FIFO queue instead of a priority queue.

Breadth-first search

```
while Q ≠ ∅
    do u ← DEQUEUE(Q)
        for each v ∈ Adj[u]
            do if d[v] = ∞
                    then d[v] ← d[u] + 1
                         ENQUEUE(Q, v)
```

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



Unweighted graphs

Suppose that $w(u, v) = 1$ for all $(u, v) \in E$.
Can Dijkstra's algorithm be improved?

- Use a simple FIFO queue instead of a priority queue.

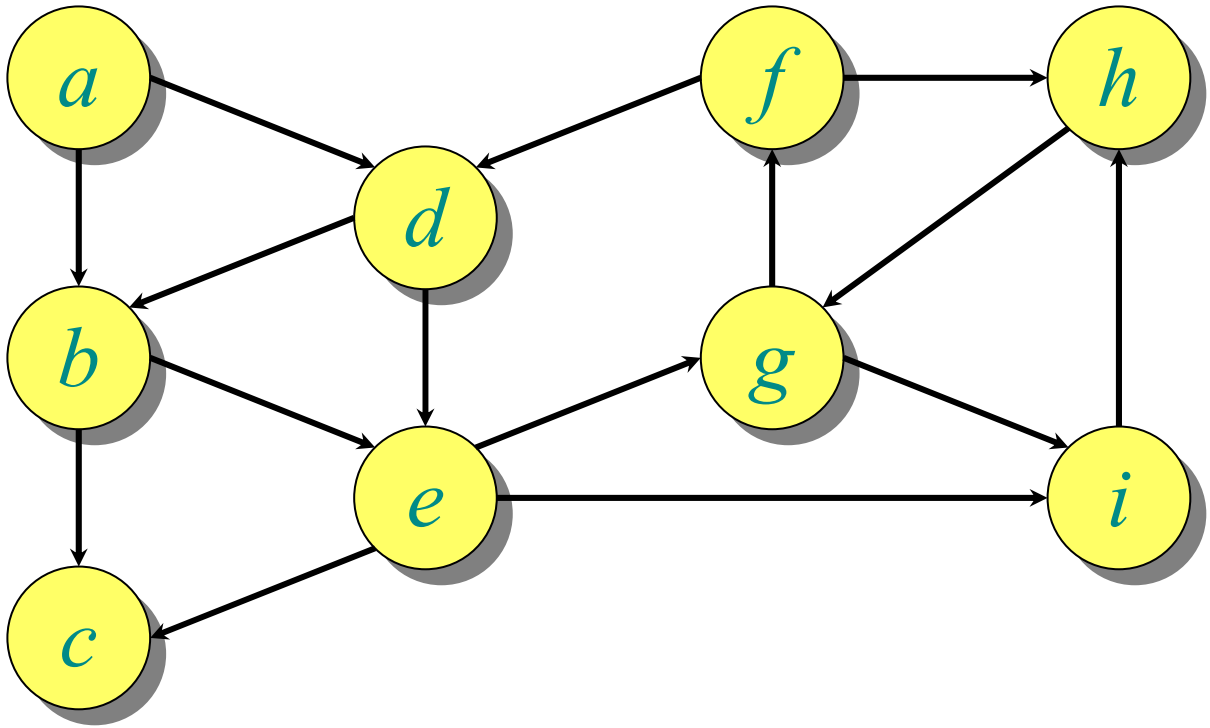
Breadth-first search

```
while Q ≠ ∅
    do u ← DEQUEUE(Q)
        for each v ∈ Adj[u]
            do if d[v] = ∞
                    then d[v] ← d[u] + 1
                         ENQUEUE(Q, v)
```

Analysis: Time $= O(V + E)$.

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

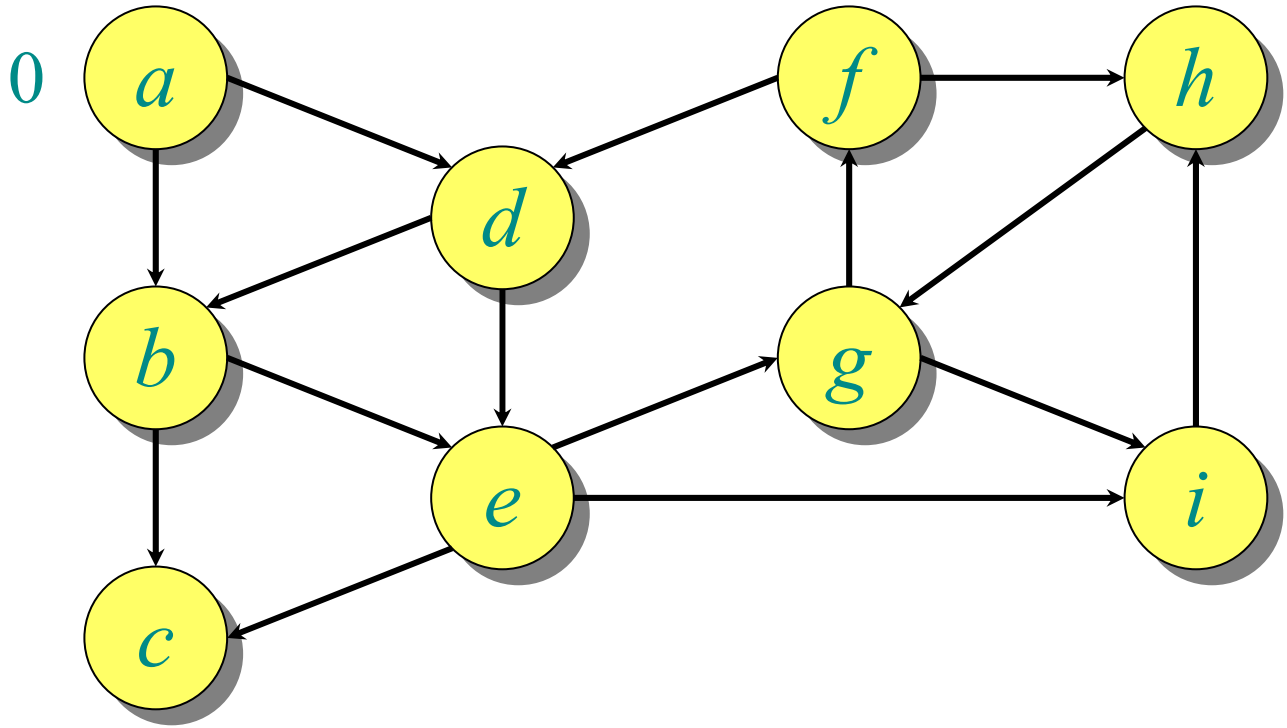
Example of breadth-first search



Q:

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

Example of breadth-first search

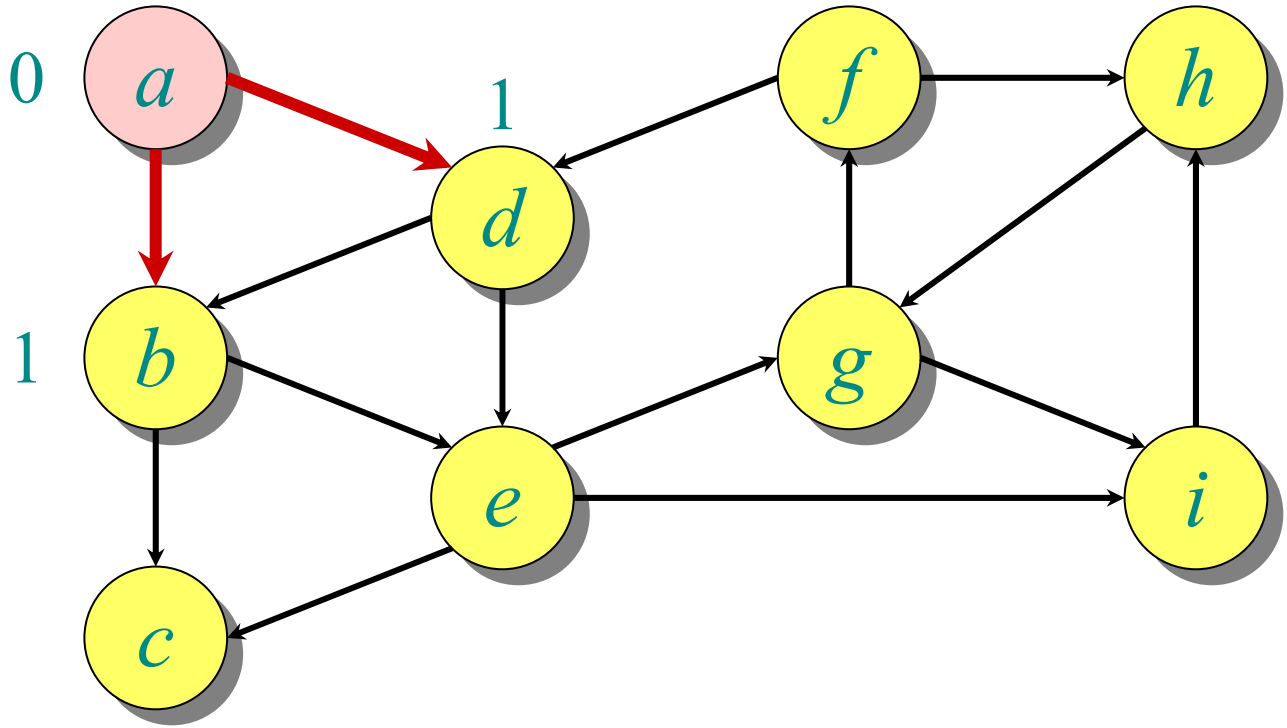


0

Q: *a*

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

Example of breadth-first search

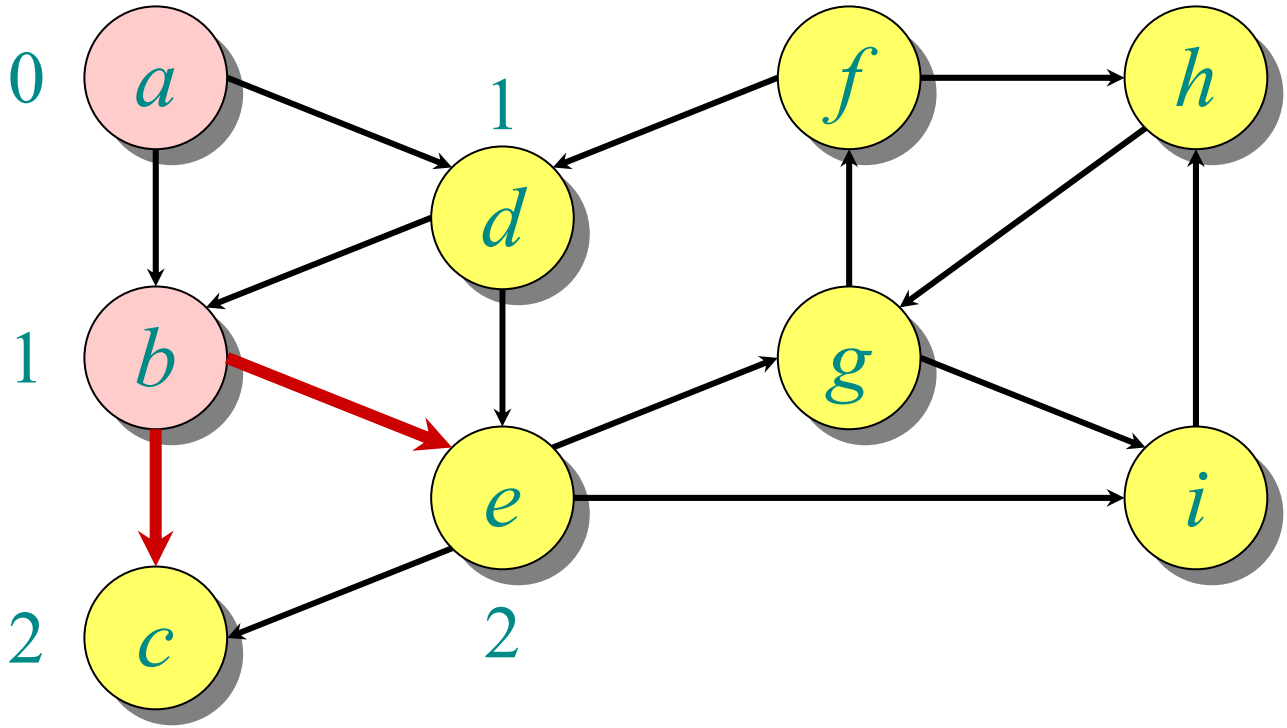


1 1

Q: *a* *b* *d*

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

Example of breadth-first search

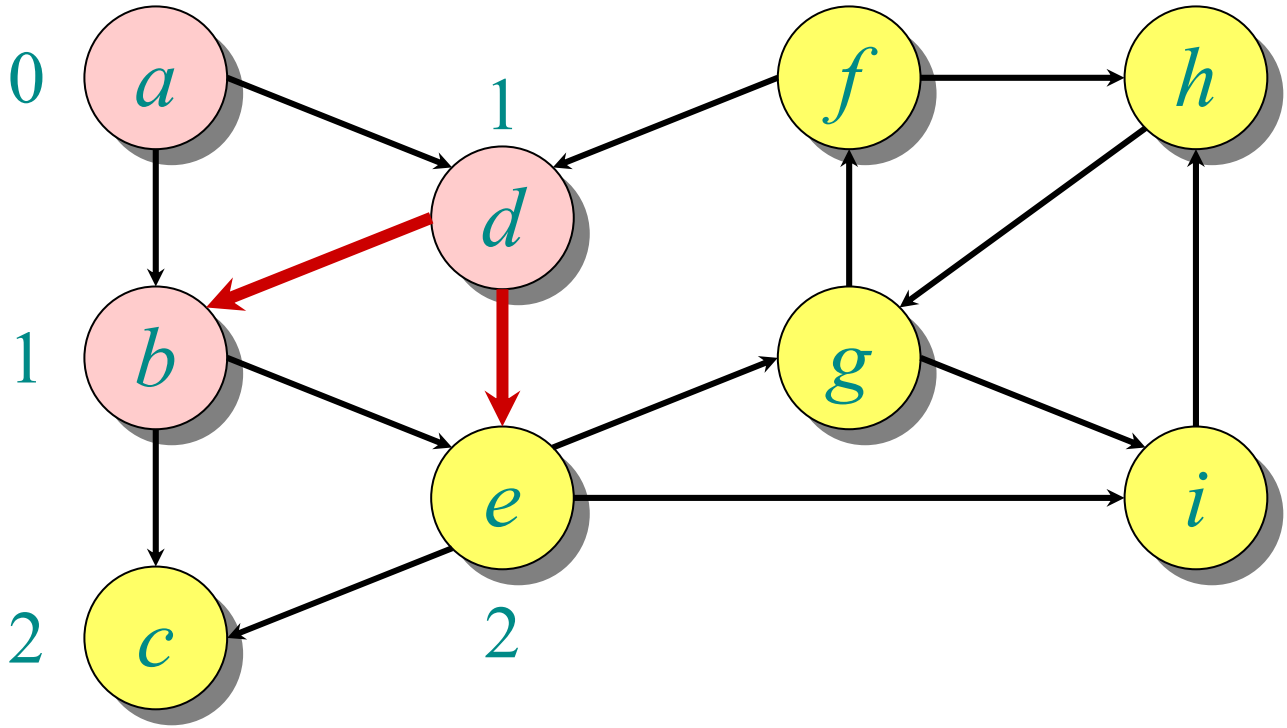


1 2 2

Q: *a* *b* *d* *c* *e*

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

Example of breadth-first search

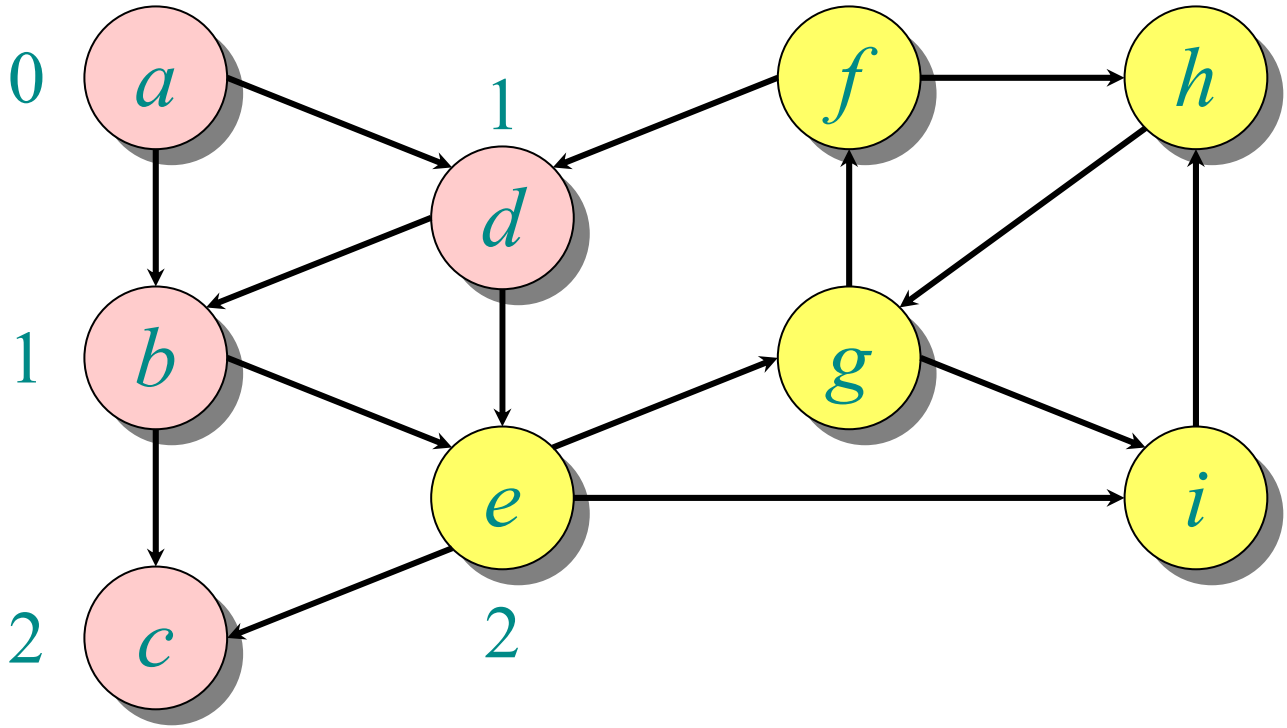


2 2

Q: *a* *b* *d* *c* *e*

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

Example of breadth-first search

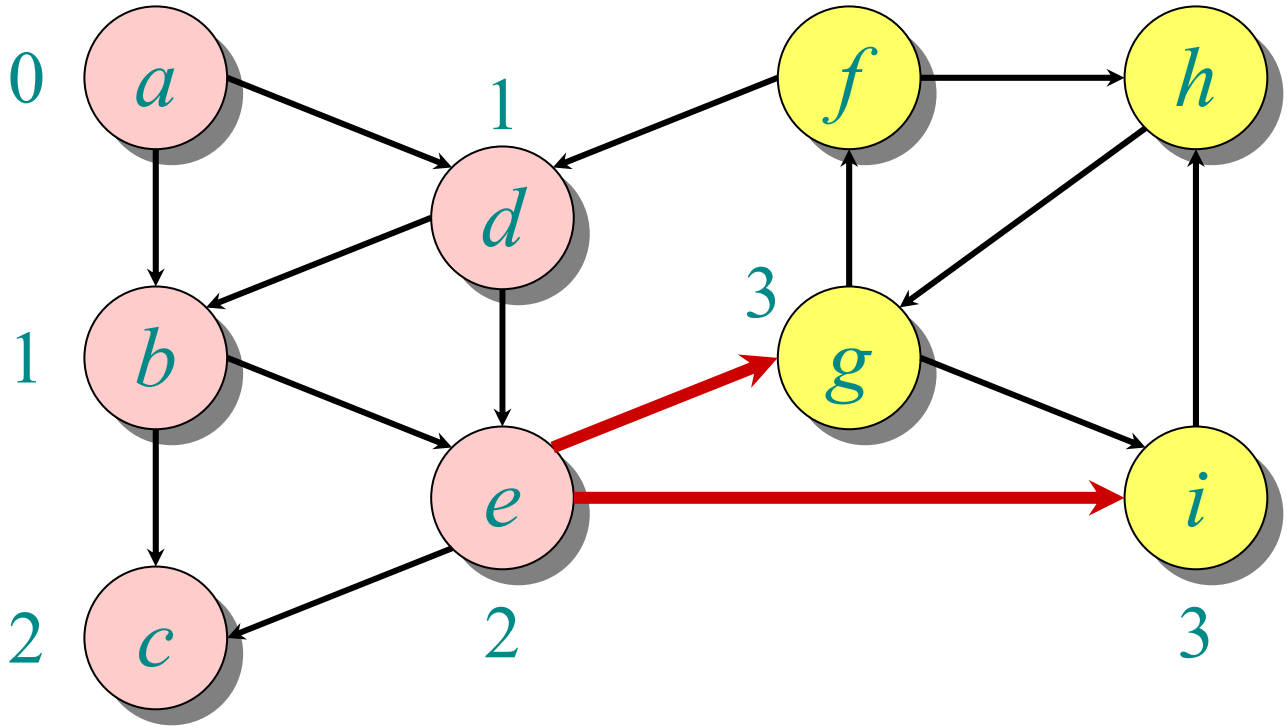


2

Q: *a* *b* *d* *c* *e*

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

Example of breadth-first search

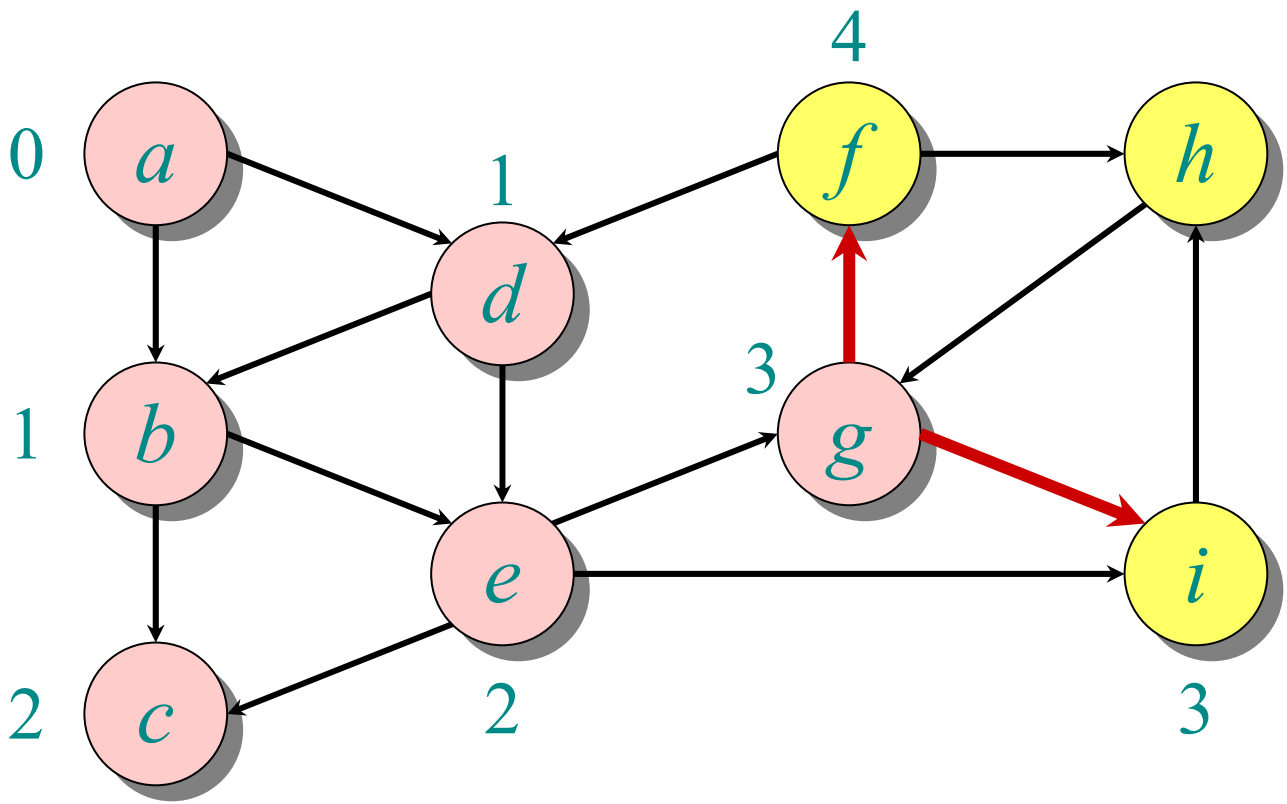


3 3

Q: *a* *b* *d* *c* *e* *g* *i*

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

Example of breadth-first search

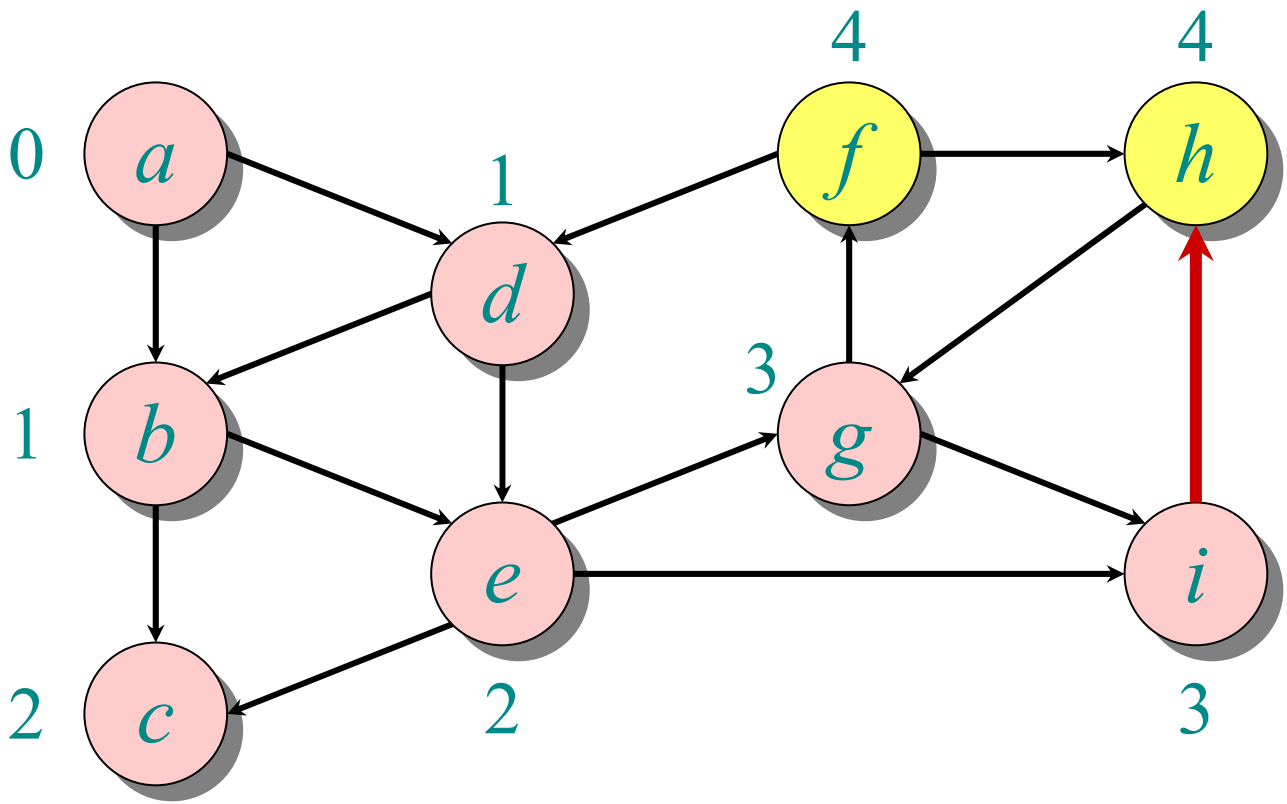


3 4

Q: *a* *b* *d* *c* *e* *g* *i* *f*

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

Example of breadth-first search

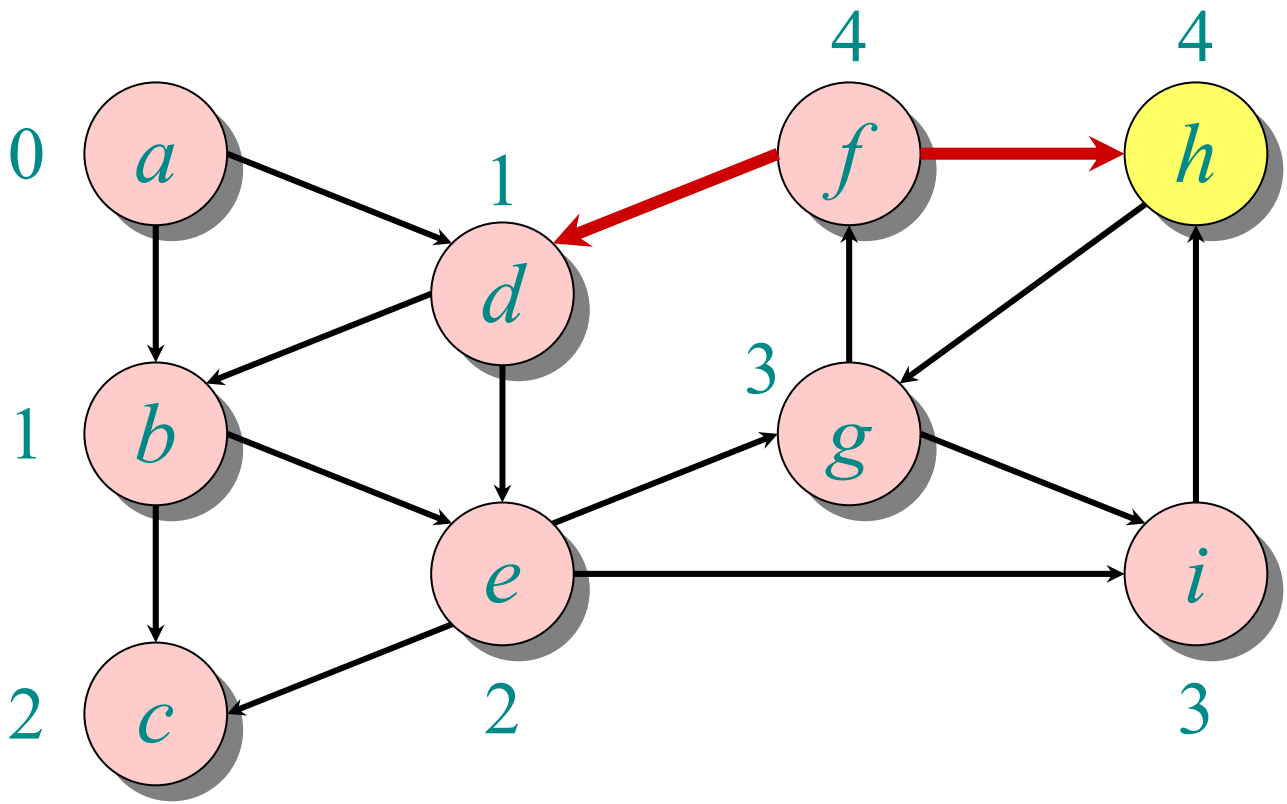


4 4

Q: *a b d c e g i f h*

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

Example of breadth-first search

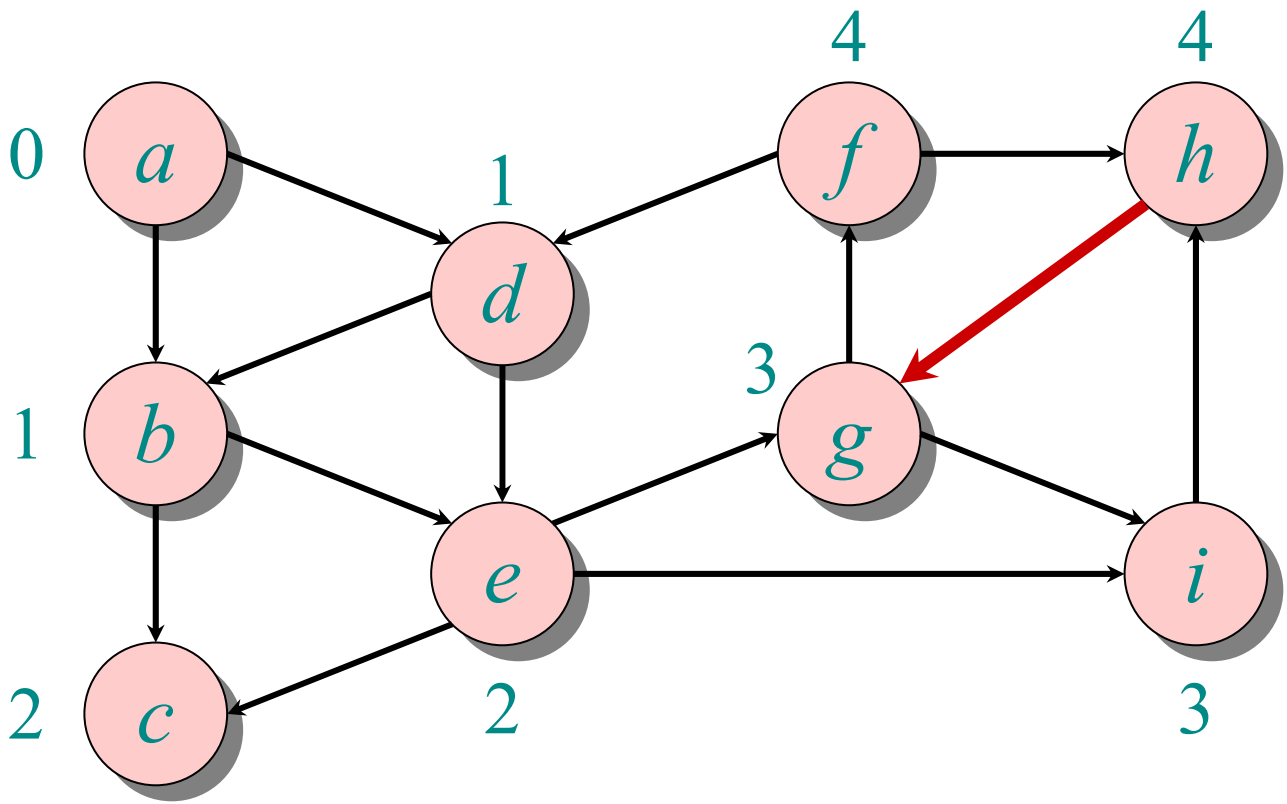


4

Q: *a b d c e g i f h*

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

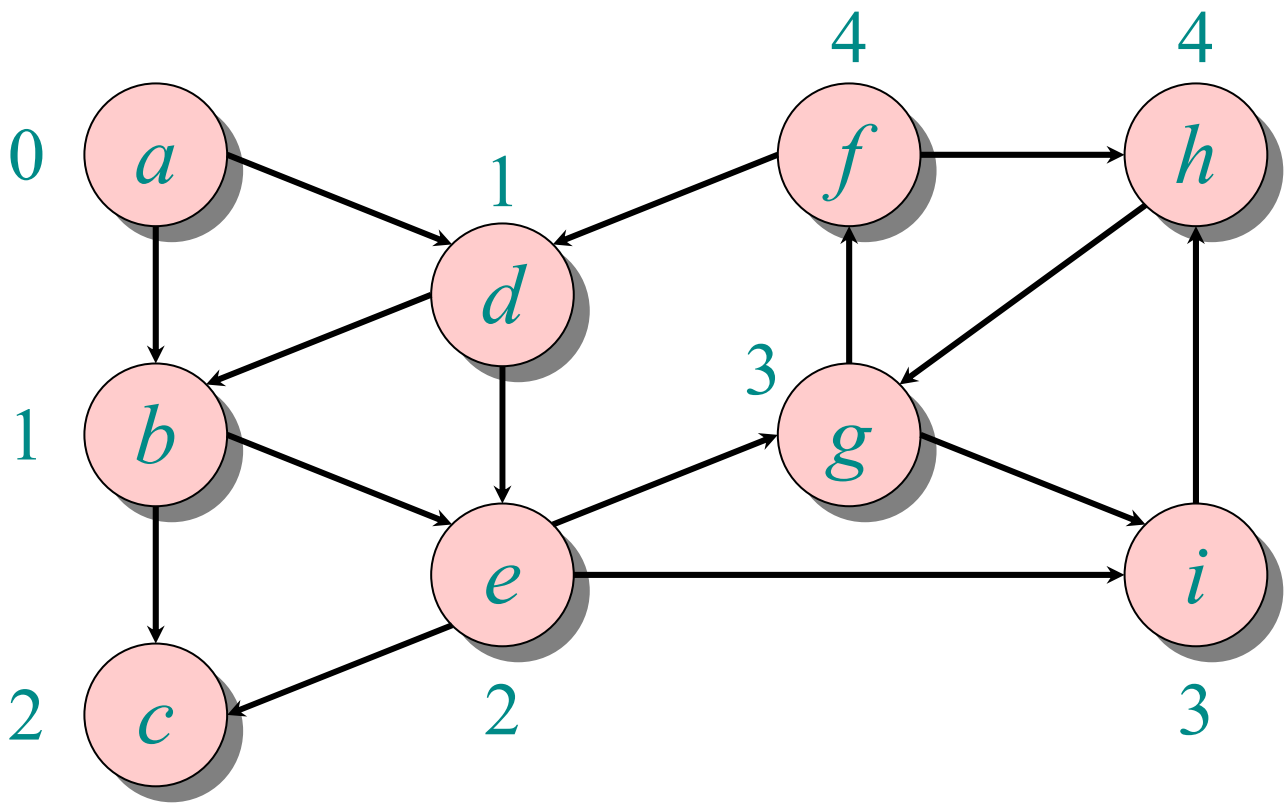
Example of breadth-first search



Q: *a b d c e g i f h*

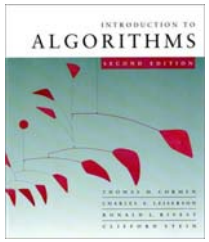
 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

Example of breadth-first search



Q: *a b d c e g i f h*

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



Correctness of BFS

```
while Q ≠ ∅
    do u ← DEQUEUE(Q)
        for each v ∈ Adj[u]
            do if d[v] = ∞
                    then d[v] ← d[u] + 1
                         ENQUEUE(Q, v)
```

Key idea:

The FIFO Q in breadth-first search mimics the priority queue Q in Dijkstra.

- **Invariant:** v comes after u in Q implies that $d[v] = d[u]$ or $d[v] = d[u] + 1$.

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson